The Agentic Engineer: Mastering Claude Code for the SDLC

Course Summary

This course provides developers and engineers with the implementation strategies and best practices for building production-ready agentic systems using Claude Code. Participants will explore advanced context management, secure integration and real-world patterns for automating the software development lifecycle. The course focuses on moving beyond basic chat-based interactions to create efficient, reliable, and autonomous workflows. While introducing an agentic framework for organizational alignment, the course emphasizes hands-on mastery of Claude Code's native features, such as custom Skills and Checkpoints.

Prerequisites:

To get the most out of this session, participants should have:

- Strong experience and programming proficiency (e.g., Python, JavaScript)
- Familiarity with command-line tools and Git workflows
- Prior foundational generative AI experience using Claude (via API or Claude.ai)
- Basic understanding of prompt engineering

Purpose	Build production-ready agentic systems using Claude Code
Audience	Core Technology Teams seeking to integrate AI-powered coding assistants safely into global workflows
Role	Software Developers | AI/ML Engineers | Technical Managers
Skill level	Intermediate
Style	Lecture | Hands-on Activities | Labs
Duration	4 days
Related technologies	AI/ML | Generative AI | Git

Learning objectives

- Set up Claude Code for daily task delegation and autonomous refactoring
- Apply context-management strategies to handle large-scale applications without data leakage
- Use MCP to securely connect agents to internal data and external APIs
- Design reliable agentic patterns to handle complex work-like unit testing and legacy migrations
- Apply production patterns for auditing AI-generated code and monitoring for security vulnerabilities

What you'll learn:

In this The Agentic Engineer: Mastering Claude Code for the SDLC course, you'll learn:

Claude Code Fundamentals

- Key differences between Claude.ai and the Claude Code CLI
  - Executing commands in a terminal-based environment
  - The shift from Generative AI (chatbots) to Agentic AI (autonomous agents)
- Configuring the CLI and managing permissions for local repository access
  - Accessing the file system and repo structure
  - Environment-specific configurations
- Core commands for planning refactors, writing code, and running tests
  - Delegating "drudge work"
    - Updating legacy code
    - Writing unit tests autonomously
  - Leveraging the agent to fix its own errors
- Using Checkpoints for experimentation and safe rollbacks of AI actions
  - Managing state snapshots during complex refactors
  - Creating a "fail-safe" environment

Context Management & Secure Integrations

- Token management strategies for 200K windows in large codebases
  - Preventing hallucinations while staying within limits
  - Optimizing context for large-scale applications
- Implementing selective inclusion and context filtering to protect PII
  - Using *.claudeignore* to avoid processing sensitive or restricted files
  - Aligning agentic deployment with security and privacy standards
- Introduction to MCP
  - Using the Model Context Protocol to connect agents to internal systems
    - Standardizing how agents access internal data silos and legacy documentation
    - Custom MCP servers to bridge AI agents and enterprise tools
- Building progressive disclosure patterns and tool-use strategies for developer agents
  - Designing "Skills" as a context optimization pattern
  - Structuring tool permissions

Agentic Workflows & Orchestration

- Creating hierarchical subagent configurations for complex engineering tasks
  - Orchestrating specialized agents
  - Structuring .md subagent configuration files
- Introduction to bMAD
  - Using the bMAD framework to structure personas
    - Defining "Analyst" and "Architect" personas
    - Applying bMAD principles
- Passing requirements and managing memory between specialized agents
  - Maintaining state and context across multi-agent workflows for large features
  - Establishing hand-off patterns between human oversight and agentic execution
- Implementing spec-driven development where humans audit AI-generated plans
  - Writing to high-level orchestration and plan review
  - Human-in-the-loop governance

Governance & Real-World Application

- Auditing and reviewing agent-produced code for security vulnerabilities
  - Developing new soft skills for reviewing "code you didn't write"
  - Ensuring safe deployment by auditing agent output
- CI/CD Integration
  - Strategies for embedding Claude Code into automated pipelines
    - Automating PR reviews and bug-fixing scripts
    - Integrating agentic verification steps into the deployment lifecycle
- Monitoring & Reliability
  - Identifying and fixing common agentic failure modes
    - Establishing patterns for debugging agent loops or incorrect context retrieval
    - Utilizing monitoring patterns to track "AI ROI" and efficiency gains

Related Pages

Custom Learning Program for Tech Skills Business Pricing for Tech Courses Top-Tier Training for Small Businesses